

Кросплатформне програмування

12

Узагальнені типи даних
(Generics)



Generics

Про що йтиметься:

- Знайомство з узагальненими класами
- Узагальнені методи
- Узагальнені класи та спадкування
- Узагальнені інтерфейси
- Узагальнені підстановки



Узагальнені класи

- В узагальнених класах тип даних є параметром
- Узагальнений клас описується як звичайний, але з формальним ідентифікатором, що позначає тип даних. Під час створення об'єкта класу ідентифікатор типу передається як параметр
- Параметри, що ототожнюються в описанні узагальненого класу з типом даних (узагальнені параметри), вказуються в кутових дужках в описанні класу після імені класу
- Під час створення об'єкта на основі узагальненого класу в кутових дужках у команді оголошення об'єктної змінної вказують ідентифікатори типу, які слід використовувати замість узагальнених параметрів
- Також ідентифікатори вказуються в кутових дужках в інструкції створення об'єкта - між назвою класу та круглими дужками з аргументами конструктора
- Значеннями узагальнених параметрів можуть бути назви класів, але не можуть бути ідентифікатори для базових типів: значеннями узагальнених параметрів ідентифікатори `int` або `char` бути не можуть, замість них використовують класи-оболонки `Integer` та `Character`



Узагальнені класи

Параметри , які ототожнюються в описанні узагальненого класу з типом даних (узагальнені параметри), вказуються у кутових дужках в описанні класу після імені класу

Описання узагальненого класу

```
class MyClass<X,Y>{  
    // Описання узагальненого класу  
}
```

У команді оголошення об'єктної змінної та в інструкції створення об'єкта у кутових дужках вказують ідентифікатори типу

Створення об'єкта узагальненого класу

```
MyClass<Integer,Character> obj=new MyClass<Integer,Character>(аргументи) ;
```

Також прийнятно використовувати спрощений синтаксис: ідентифікатори типів у команді виклику конструктора можна не вказувати (але при цьому кутові дужки мають бути присутніми)

Створення об'єкта узагальненого класу

```
MyClass<Integer,Character> obj=new MyClass<>(аргументи) ;
```



Узагальнені класи

```
class MyClass<X>{           // Узагальнений клас із одним параметром типу
    X data;                 // Поле узагальненого типу
    MyClass(X data){        // Конструктор з аргументом узагальненого типу
        this.data=data;     // Привласнення значення полю
    }
    void show(){            // Метод для відображення значення поля
        System.out.println("Значення поля: "+data);
    }
} // Головний клас:
class Demo{
    public static void main(String[] args){
        // Створення об'єкта на основі узагальненого класу.
        // Замість узагальненого параметра використовується клас Integer:
        MyClass<Integer> A=new MyClass<Integer>(100);
        // Створення об'єкта на основі узагальненого класу.
        // Замість узагальненого параметра використовується клас Character:
        MyClass<Character> B=new MyClass<Character>('B');
        // Створення об'єкта на основі узагальненого класу.
        // Замість узагальненого параметра використовується клас String:
        MyClass<String> C=new MyClass<String>("Зелений");
        // Виклик методу show() з об'єктів узагальненого класу:
        A.show();
        B.show();
        C.show();
    }
}
```

Результат:

Значення поля: 100

Значення поля: B

Значення поля: Зелений



Узагальнені методи

- Крім узагальнених класів, можна створювати узагальнені методи
- В описанні узагальненого методу тип визначається через узагальнений параметр (або параметри), а при виклику методу значення узагальнених параметрів визначаються на основі типів аргументів, переданих методу
- Зазвичай (але не обов'язково) узагальнені методи статичні

Описання статичного узагальненого методу

```
static <параметр> тип_результату ім'я_методу(аргументи) {  
    // Код методу  
}
```

Описання нестатичного узагальненого методу

```
<параметр> тип_результату ім'я_методу(аргументи) {  
    // Код методу  
}
```



Статичні узагальнені методи

```
// Клас зі статичними узагальненими методами:
class Demo{
    // Метод з аргументом узагальненого типу:
    static <X> void show(X arg){
        System.out.println(arg);
    }
    // Аргумент методу - узагальнений масив:
    static <X> void show(X[] array){
        System.out.print("|");
        // Відображення значень елементів масиву:
        for(int k=0;k<array.length;k++){
            System.out.print(array[k]+" | ");
        }
        System.out.println();
    }
    // Методу аргументом передається узагальнений масив та
    // цілочисловий індекс, а результатом повертається
    // значення елемента з відповідним індексом:
    static <X> X getElement(X[] array,int index){
        return array[index];
    }
    // Продовження на наступному слайді!
```



Статичні узагальнені методи

```
public static void main(String[] args){           // Головний метод
    Integer[] nums={1,3,7,2};                     // Цілочисловий масив
    Character[] syms={'A','W','L','O','B'};        // Символьний масив
    System.out.println("Виклик методу show()");
    System.out.print("З текстовим аргументом: ");
    show("узагальнений метод");
    System.out.print("З int-аргументом: ");
    show(123);
    System.out.print("З double-аргументом: ");
    show(123.45);
    System.out.print("З char-аргументом: ");
    show('A');
    System.out.print("Цілочисловий масив: ");
    show(nums);
    System.out.print("Символьний масив: ");
    show(syms); // Поелементне відображення масивів:
    System.out.println("Виклик методу getElement()");
    System.out.print("Цілочисловий масив: *");
    for(int k=0;k<nums.length;k++){
        System.out.print(getElement(nums,k)+"*");
    }
    System.out.print("\nСимвольний масив: *");
    for(int k=0;k<syms.length;k++){
        System.out.print(getElement(syms,k)+"*");
    }
    System.out.println();
}}
```




Статичні узагальнені методи

Результат:

Виклик методу show()

З текстовим аргументом: узагальнений метод

З int-аргументом: 123

З double-аргументом: 123.45

З char-аргументом: A

Цілочисловий масив: | 1 | 3 | 7 | 2 |

Символьний масив: | A | W | L | O | B |

Виклик методу getElement()

Цілочисловий масив: *1*3*7*2*

Символьний масив: *A*W*L*O*B*



Нестатичні узагальнені методи

Ми можемо описати узагальнений клас, у якому використовується метод із узагальненими параметрами типу. Але можна зробити й інакше: клас описувати як звичайний, але при цьому один або більше методів описати як узагальнені

```
// Клас із узагальненим методом:
class MyClass{
    String name;                // Текстове поле
    <X> void show(X arg){        // Узагальнений метод
        System.out.println(name+": "+arg);
    }
    MyClass(String txt){        // Конструктор
        name=txt;
    }
} // Головний клас:
class Demo{
    public static void main(String[] args){ // Головний метод
        MyClass A=new MyClass("Об'єкт А"); // Створення об'єктів
        MyClass B=new MyClass("Об'єкт В");
        A.show(123);             // Виклик узагальнених методів із різних об'єктів
        A.show("Alpha");
        A.show('A');
        B.show(123);
        B.show("Bravo");
        B.show('B');
    }
}
```

Результат:

```
Об'єкт А: 123
Об'єкт А: Alpha
Об'єкт А: A
Об'єкт В: 123
Об'єкт В: Bravo
Об'єкт В: B
```



Узагальнені класи та спадкування

```
class Base<X>{
    // Узагальнене поле:
    X data;
    // Конструктор:
    Base(X data){
        this.data=data;
    } // Метод для відображення значення поля:
    void show(){
        System.out.println(data);
    }
}
// Підклас на основі узагальненого класу з цілочисловим
// типом замість узагальненого:
class Alpha extends Base<Integer>{
    // Конструктор:
    Alpha(Integer n){
        // Виклик конструктора суперкласу:
        super(n);
    } // Переозначення методу:
    void show(){
        System.out.print("Цілочислове поле: ");
        // Виклик версії методу з суперкласу:
        super.show();
    }
} // Продовження на наступному слайді!
```



Узагальнені класи та спадкування

```
class Bravo extends Base<String>{
    // Конструктор:
    Bravo(String txt){
        // Виклик конструктора суперкласу:
        super(txt);
    } // Переозначення методу:
    void show(){
        System.out.print("Текстове поле: ");
        // Виклик версії методу з суперкласу:
        super.show();
    }
}
// Підклас узагальненого класу з символьним типом замість узагальненого:
class Charlie extends Base<Character>{
    // Конструктор:
    Charlie(Character s){
        // Виклик конструктора суперкласу:
        super(s);
    } // Переозначення методу:
    void show(){
        System.out.print("Символьне поле: ");
        // Виклик версії методу з суперкласу:
        super.show();
    }
} // Продовження на наступному слайді!
```

Узагальнені класи та спадкування

```
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Створення об'єктів:
        Alpha A=new Alpha(123);
        Bravo B=new Bravo("об'єкт В");
        Charlie C=new Charlie('C');
        // Виклик методу show() з різних об'єктів:
        A.show();
        B.show();
        C.show();
    }
}
```

Результат:

Цілочислове поле: 123
Текстове поле: об'єкт В
Символьне поле: C



Обмеження спадкування

В узагальнених класах і методах в інструкції оголошення узагальненого типу може використовуватися вираз вигляду **X extends Суперклас**, який означає, що узагальнений тип **X** повинен бути підкласом (прямим або опосередкованим) класу **Суперклас**

```
class Base{                                // Суперклас
    String name;                            // Текстове поле
    Base(String txt){                      // Конструктор
        name=txt;
    } // Метод для відображення значення поля:
    void show(){
        System.out.println("Текстове поле: "+name);
    }
} // Підклас суперкласу Base:
class Alpha extends Base{
    int number;                            // Цілочислове поле
    Alpha(String txt,int n){               // Конструктор
        super(txt);                       // Виклик конструктора суперкласу
        number=n;                         // Привласнення значення цілочисловому полю
    } // Переозначення методу:
    void show(){
        super.show();                     // Виклик версії методу з суперкласу
        // Відображення значення цілочислового поля:
        System.out.println("Цілочислове поле: "+number);
    }
} // Продовження на наступному слайді!
```



Обмеження спадкування

```
// Підклас суперкласу Alpha:
class Bravo extends Alpha{
    // Символьне поле:
    char symbol;
    // Конструктор:
    Bravo(String txt,int n,char s){
        // Виклик конструктора суперкласу:
        super(txt,n);
        // Присвоюється значення символічному полю:
        symbol=s;
    }
    // Переозначення методу:
    void show(){
        // Виклик версії методу з суперкласу:
        super.show();
        // Відображення значення символічного поля:
        System.out.println("Символьне поле: "+symbol);
    }
}
// Продовження на наступному слайді!
```



Обмеження спадкування

```
// Узагальнений клас:
class MyClass<X extends Base>{
    // Поле узагальненого типу:
    X obj;
    // Конструктор:
    MyClass(X obj){
        // Значення поля:
        this.obj=obj;
    } // Метод узагальненого класу:
    void show(){
        System.out.println("Об'єкт класу MyClass");
        // Виклик методу з поля узагальненого типу:
        obj.show();
    }
} // Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Об'єкти створюються на основі узагальненого класу:
        MyClass<Alpha> A=new MyClass<>(new Alpha("Alpha",123));
        MyClass<Bravo> B=new MyClass<>(new Bravo("Bravo",321,'B'));
        // Виклик методу show() з об'єктів:
        A.show();
        B.show();
    }
}
```

Результат:

Об'єкт класу MyClass
Текстове поле: Alpha
Цілочислове поле: 123
Об'єкт класу MyClass
Текстове поле: Bravo
Цілочислове поле: 321
Символьне поле: B



Узагальнені інтерфейси

- Узагальненим може бути не тільки клас або метод, а й інтерфейс
- Оголошується узагальнений інтерфейс аналогічно до узагальненого класу: після імені інтерфейсу в кутових дужках вказують назви для узагальнених типів
- Узагальнений інтерфейс може використовуватися як для створення узагальненого класу, так і для створення класу з конкретними значеннями для узагальнених параметрів типу

```
interface MyMethods<X>{           // Узагальнений інтерфейс
    X get();
    void set(X arg);
} // Створення узагальненого класу на основі узагальненого інтерфейсу:
class MyClass<X> implements MyMethods<X>{
    private X value;              // Закрите поле узагальненого типу
    public X get(){               // Описання методів з інтерфейсу
        return value;
    }
    public void set(X arg){
        value=arg;
    } // Метод для відображення значення поля:
    void show(){
        System.out.println("Значення поля: "+get());
    } // Конструктор:
    MyClass(X arg){
        value=arg;
    }
} // Продовження на наступному слайді!
```



Узагальнені інтерфейси

```
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Інтерфейсна змінна:
        MyMethods ref;
        // Створення об'єктів на основі узагальненого класу:
        MyClass<Integer> A=new MyClass<>(123);
        MyClass<Character> B=new MyClass<>('A');
        // Виклик методу з об'єкта класу:
        A.show();
        // Інтерфейсній змінній присвоюється значення:
        ref=A;
        // Виклик методу через інтерфейсну змінну:
        ref.set(321);
        // Виклик методів з об'єктів класу:
        A.show();
        B.show();
        // Інтерфейсній змінній присвоюється значення:
        ref=B;
        // Виклик методу через інтерфейсну змінну:
        ref.set('B');
        // Виклик методу з об'єкта класу:
        B.show();
    }
}
```

Результат:

Значення поля: 123

Значення поля: 321

Значення поля: A

Значення поля: B



Узагальнені інтерфейси

```
interface MyMethods<X>{
    X get();
    void set (X arg);
}
// Створення першого класу на основі
// узагальненого інтерфейсу:
class Alpha implements MyMethods<Integer>{
    // Закрите поле цілого типу:
    private Integer value;
    // Опис методів з інтерфейсу:
    public Integer get(){
        return value;
    }
    public void set(Integer arg){
        value=arg;
    }
    // Метод для відображення значення поля:
    void show(){
        System.out.println("Цілочислове поле: "+get());
    }
    // Конструктор:
    Alpha(Integer arg){
        value=arg;
    }
} // Продовження на наступному слайді!
```



Узагальнені інтерфейси

```
// Створення другого класу на основі
// узагальненого інтерфейсу:
class Bravo implements MyMethods<Character>{
    // Закрите поле символьного типу:
    private Character value;
    // Описання методів з інтерфейсу:
    public Character get(){
        return value;
    }
    public void set(Character arg){
        value=arg;
    }
    // Метод для відображення значення поля:
    void show(){
        System.out.println("Символьне поле: "+get());
    }
    // Конструктор:
    Bravo(Character arg){
        value=arg;
    }
}
// Продовження на наступному слайді!
```



Узагальнені інтерфейси

```
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Інтерфейсна змінна:
        MyMethods R;
        // Створення об'єктів:
        Alpha A=new Alpha(123);
        Bravo B=new Bravo('A');
        // Виклик методу з об'єкта класу:
        A.show();
        // Інтерфейсною змінною надається значення:
        R=A;
        // Виклик методу через інтерфейсну змінну:
        R.set(321);
        // Виклик методів з об'єктів класу:
        A.show();
        B.show();
        // Інтерфейсній змінній присвоюється значення:
        R=B;
        // Виклик методу через інтерфейсну змінну:
        R.set('B');
        // Виклик методу з об'єкта класу:
        B.show();
    }
}
```

Результат:

Цілочислове поле: 123

Цілочислове поле: 321

Символьне поле: A

Символьне поле: B

Узагальнені інтерфейси

Інтерфейс `MyMethods` є узагальненим. Інтерфейсна змінна `R` оголошується командою `MyMethods R`, в якій вказано лише ім'я інтерфейсу без вказування значення для узагальненого типу. Завдяки цьому значенням змінній `R` можна присвоїти посилання як на об'єкт `A`, так і на об'єкт `B` - незважаючи на те, що об'єкт `A` створювався на основі класу `Alpha`, що реалізує інтерфейс `MyMethods` зі значенням `Integer` для узагальненого типу, а об'єкт `B` створюється на основі класу `Bravo`, що реалізує інтерфейс `MyMethods` зі значенням `Character` для узагальненого типу. Така "універсальність" має ціну: узагальнений тип при зверненні до об'єктів через інтерфейсну змінну `R` не специфікований. Тому, наприклад, метод `get()`, який відповідно до свого оголошення в інтерфейсі, повертає значення узагальненого типу, а для об'єктів `A` та `B` описаний таким чином, що повертає значення типу `Integer` та `Character`, при виклику його через змінну `R` інтерпретується як таке, що повертає значення типу `Object`. Це не заважає, скажімо, відображати в консольному вікні значення, що повертається методом `get()`, але накладає обмеження на допустимі операції з результатом методу (щоправда, зазвичай, на виручку може прийти явне приведення типу).

З іншого боку, якби ми, скажімо, оголосити змінну `R` з допомогою команди `MyMethods<Integer> R`, у якій для інтерфейсу `MyMethods` явно вказано значення `Integer` для параметра узагальненого типу. Але в такому разі значенням змінної `R` можна було б присвоїти лише посилання на об'єкт `A`, але не на об'єкт `B`. Проте під час виклику методу `get()` через змінну `R` результат методу інтерпретується як значення типу `Integer`.



Узагальнені підстановки

Ідея узагальненої підстановки в тому, що при визначенні узагальненого параметра використовують інструкцію вигляду `<?>`. Це повідомлення для компілятора, що у відповідному місці використовується деякий невідомий узагальнений тип

Реалізуємо таку програму:

У програмі визначено узагальнений клас `MyClass` з одним узагальненим параметром (позначимо його як `T`)

У класі оголошено поле `value` узагальненого типу `T` та конструктор з одним аргументом узагальненого типу. Аргумент конструктора присвоюється значенням полю `value`

У класі `Demo` описується головний метод `main()` і два статичні методи, що не повертають результат: `show()` та `display()`. Вони виконують практично однакову "роботу", але описані по-різному

Обидва методи призначені для відображення значення поля `value` об'єкта, створеного на основі узагальненого класу `MyClass` та переданого аргументом методу

У головному методі програми наведено приклади створення об'єктів на основі узагальненого класу `MyClass` з подальшою їх передачею аргументами методам `show()` та `display()`



Узагальнені підстановки

```
// Узагальнений клас:
class MyClass<T>{
    // Поле узагальненого типу:
    T value;
    // Конструктор:
    MyClass(T val){
        value=val;
    }
}
// Головний клас:
class Demo{
    // Статичний узагальнений метод
    // (використовується параметр узагальненого типу):
    static <T> void show(MyClass<T> obj){
        System.out.println("Виклик методу show():");
        // Відображення значення поля:
        System.out.println(obj.value);
    }
    // Статичний метод, у якому використовується
    // узагальнена підстановка:
    static void display(MyClass<?> obj){
        System.out.println("Виклик методу display():");
        // Відображення значення поля:
        System.out.println(obj.value);
    }
}
// Продовження на наступному слайді!
```




Узагальнені підстановки

```
// Головний метод:
public static void main(String[] args){
    // Під час створення об'єкта явно вказано значення
    // для узагальненого типу:
    MyClass<Integer> A=new MyClass<>(100);
    // При створенні об'єкта не вказано значення
    // для узагальненого типу:
    MyClass B=new MyClass<>('B');
    // Під час створення об'єкта використана
    // узагальнена підстановка:
    MyClass<?> C=new MyClass<>("Об'єкт C");
    // Приклади виклику методів show() та display()
    // з різними аргументами:
    show(A);
    display(A);
    show(B);
    display(B);
    show(C);
    display(C);
}
```

У всіх трьох випадках ми
можемо відобразити значення
поля, але операції, допустимі
для виконання з полем value,
залежать від способу
створення об'єкта і різні для
об'єктів A, B та C

Результат:

```
Виклик методу show() :
100
Виклик методу display() :
100
Виклик методу show() :
В
Виклик методу display() :
В
Виклик методу show() :
Об'єкт C
Виклик методу display() :
Об'єкт C
```

Узагальнені підстановки

Різниця між способами оголошення об'єктів А, В та С:

- Під час створення об'єкта А ми явно вказуємо значення `Integer` для параметра узагальненого типу, тому поле `value` створеного об'єкта інтерпретується як `Integer-значення`
- Під час створення об'єкта В для об'єктної змінної вказано лише основний тип (ім'я класу `MyClass`). Тому поле `value` створеного об'єкта інтерпретується як значення типу `Object`
- Під час створення об'єкта С використано узагальнену підстановку. Для цього об'єкта значення узагальненого параметра не ідентифікується. Технічно в такому випадку використовується формальне "робоче" позначення для параметра узагальненого типу - тобто для параметра узагальненого типу виконується "підстановка", яка грає роль типу. Такі підстановки використовуються, якщо виконання коду не залежить від значення узагальненого параметра



Узагальнені підстановки з обмеженнями

У разі використання узагальнених підстановок можна використовувати обмеження: за допомогою ключового слова **extends** визначають підстановки, що позначають підкласи для даного класу, а за допомогою ключового слова **super** визначають підстановки для суперкласів даного класу

- Вираз виду **<? extends SuperClass>** означає, що для параметра узагальненого типу використовується невстановлений тип, що є підкласом (прямим або опосередкованим) класу **SuperClass**
- Вираз виду **<? super SubClass>** означає, що як параметр узагальненого типу може бути клас, що є (прямо або через ланцюжок спадкування) суперкласом для класу **SubClass**



Узагальнені підстановки з обмеженнями

```
class Alpha{
    // Закрите текстове поле:
    private String name;
    // Конструктор:
    Alpha(String txt){
        name=txt;
    }
    // Переозначення методу toString():
    public String toString(){
        return name;
    }
} // Другий клас:
class Bravo extends Alpha{
    // Конструктор:
    Bravo(String txt){
        // Виклик конструктора суперкласу:
        super(txt);
    }
} // Третій клас:
class Charlie extends Bravo{
    Charlie(String txt){
        super(txt);
    }
}
// Продовження на наступному слайді!
```



Узагальнені підстановки з обмеженнями

```
// Четвертий клас:
class Delta extends Charlie{
    Delta (String txt){
        super(txt);
    }
}
// П'ятий клас:
class Echo extends Delta{
    Echo(String txt){
        super(txt);
    }
}
// Узагальнений клас:
class MyClass<T>{
    // Закрите поле узагальненого типу:
    private T obj;
    // Перевизначення методу toString():
    public String toString(){
        return obj.toString();
    }
    // Конструктор:
    MyClass(T arg){
        obj=arg;
    }
}
// Продовження на наступному слайді!
```



Узагальнені підстановки з обмеженнями

```
class Demo{    // Головний клас
    // Статичний метод для відображення текстового представлення об'єкта,
    // створеного з використанням підкласу для класу Charlie:
    static void show(MyClass<? extends Charlie> obj){
        System.out.println(obj);
    } // Статичний метод для відображення текстового представлення об'єкта,
    // створеного з використанням суперкласу для класу Charlie:
    static void display(MyClass<? super Charlie> obj){
        System.out.println(obj);
    } // Головний метод:
    public static void main(String[] args){ // Створення об'єктів:
        MyClass<Alpha> A=new MyClass<>(new Alpha("Об'єкт A"));
        MyClass<Bravo> B=new MyClass<>(new Bravo("Об'єкт B"));
        MyClass<Charlie> C=new MyClass<>(new Charlie("Об'єкт C"));
        MyClass<Delta> D=new MyClass<>(new Delta("Об'єкт D"));
        MyClass<Echo> E=new MyClass<>(new Echo("Об'єкт E"));
        // Виклик методів display() та show() з передачею
        // аргументом одного зі створених об'єктів:
        display(A);
        display(B);
        display(C);
        show(C);
        show(D);
        show(E);
    }
}
```

Результат :

Об'єкт A
Об'єкт B
Об'єкт C
Об'єкт C
Об'єкт D
Об'єкт E

Дякую за увагу!

Запитання?